

Data Warehousing

Lecture- 5: DW Modelling (Types of Dimensions)

Ali Zidane ElQutaany

Types of Dimensions

- **TEN Types of Dimension Tables:**
 1. Slowly Changing Dimensions.
 2. Conformed Dimensions (Distorted Dimension)
 3. Degenerate Dimensions.
 4. Junk Dimensions.
 5. Role Playing Dimensions.
 6. Static Dimensions (Fixed Dimensions)
 7. Shrunk Dimensions (Foreign Dimensions Manager).
 8. Stacked Dimensions.
 9. Deferred Dimension (Sensible Dimension).
 10. Rapidly Changing Dimensions.

Slowly Changing Dimensions

- **Slowly Changing Dimensions:**

1. **Type 0 - The passive method** (some values are overwritten (**Updated ONES**), and others remain the same). Some records are updated based on CDC (Capture Data Change)
2. **Type 1 - Overwriting the old value.** In this method no history of dimension changes is kept. The old dimension value is simply overwritten by the new one. This type is easy to maintain and is often use for data which changes are caused by processing corrections(e.g., removal special characters, correcting spelling errors).

Before the change:

Customer_ID	Customer_Name	Customer_Type
1	Cust_1	Corporate

After the change:

Customer_ID	Customer_Name	Customer_Type
1	Cust_1	Retail

Slowly Changing Dimensions

- **Slowly Changing Dimensions:**

- 3. **Type 2** - Creating a new additional record:

- In this methodology all history of dimension changes is kept.
 - You capture attribute change by adding a new row with a new surrogate key to the dimension table. Both the prior and new rows contain as attributes the natural key(or another durable identifier).
 - Also, **Effective Date** and 'current flag' columns are used in this method. There could be only one record with current indicator set to 'Y'. For 'effective date' columns, i.e., Start Date and End Date, the End Date for current record usually is set to value 9999-12-31.
 - Introducing changes to the dimensional model in type 2 could be very expensive database operation so it is not recommended to use it in dimensions where a new attribute could be added in the future.

Slowly Changing Dimensions

- Slowly Changing Dimensions: Type 2 -** Creating a new additional record:

Before the change:

Customer_ID	Customer_Name	Customer_Type	Start_Date	End_Date	Current_Flag
1	Cust_1	Corporate	22-07-2010	31-12-9999	Y

After the change:

Customer_ID	Customer_Name	Customer_Type	Start_Date	End_Date	Current_Flag
1	Cust_1	Corporate	22-07-2010	17-05-2012	N
2	Cust_1	Retail	18-05-2012	31-12-9999	Y

Slowly Changing Dimensions

- **Slowly Changing Dimensions:**

- 4. **Type 3** - Adding a new column:

- In this type usually only the current and previous value of dimension is kept in the database. The new value is loaded into 'current/new' column and the old one into 'old/previous' column.
 - The history is limited to the number of column created for storing historical data. This is the least commonly needed technique.

Slowly Changing Dimensions

- **Slowly Changing Dimensions: Type 3** - Adding a new column:

Before the change:

Customer_ID	Customer_Name	Current_Type	Previous_Type
1	Cust_1	Corporate	Corporate

After the change:

Customer_ID	Customer_Name	Current_Type	Previous_Type
1	Cust_1	Retail	Corporate

Slowly Changing Dimensions

- **Slowly Changing Dimensions:**

- 5. **Type 4** - Using historical table.

- The 'main' dimension table keeps only the current data e.g. customer and customer history tables.
 - In this method a separate historical table is used to track all dimension's attribute historical changes for each of the dimension.

Slowly Changing Dimensions

- **Slowly Changing Dimensions: Type 4** - Using historical table.

Current table:

Customer_ID	Customer_Name	Customer_Type
1	Cust_1	Corporate

Historical table:

Customer_ID	Customer_Name	Customer_Type	Start_Date	End_Date
1	Cust_1	Retail	01-01-2010	21-07-2010
1	Cust_1	Oher	22-07-2010	17-05-2012
1	Cust_1	Corporate	18-05-2012	31-12-9999

Slowly Changing Dimensions

- **Slowly Changing Dimensions:**

Type 6 - Combine approaches of types 1,2,3 ($1+2+3=6$). In this type we have in dimension table such additional columns as:

- **Current Type** - for keeping current value of the attribute. All history records for given item of attribute have the same current value.
- **Historical Type** - for keeping historical value of the attribute. All history records for given item of attribute could have different values.
- **StartDate**- for keeping start date of 'effective date' of attribute's history.
- **End Date**- for keeping end date of 'effective date' of attribute's history.
- **Current Flag**- for keeping information about the most recent record.
- In this method to capture attribute change we add a new record as in type 2. The **Current Type** information is overwritten with the new one as in type 1. We store the history in a **Historical Column** as in type 3.

Slowly Changing Dimensions

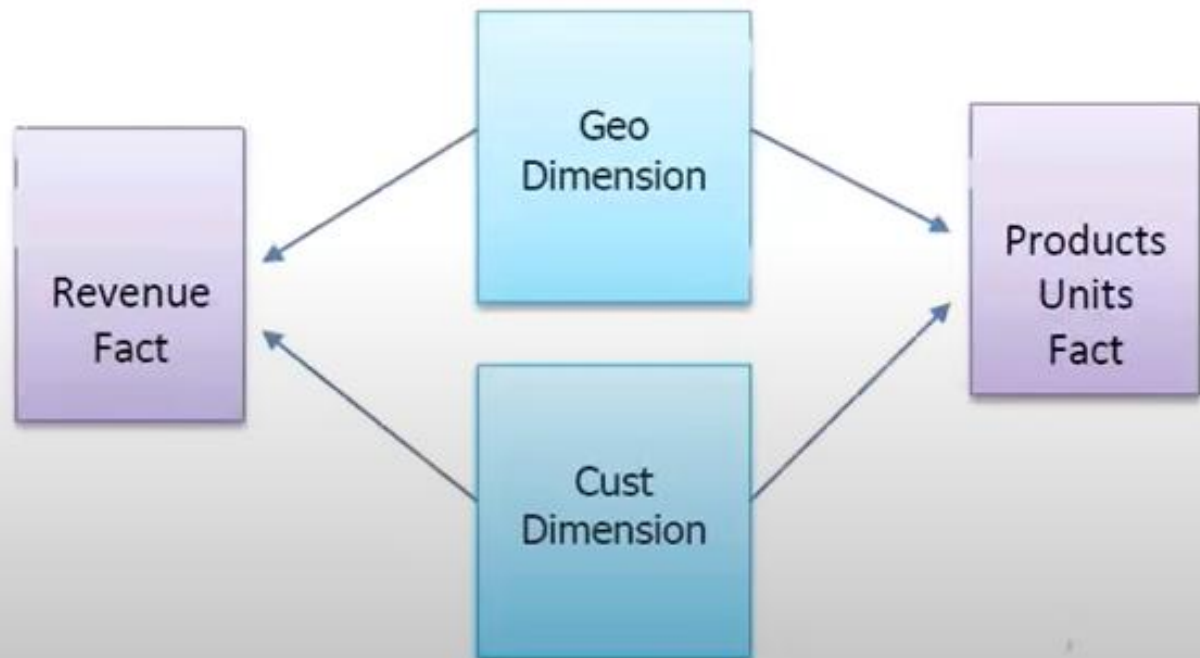
- **Slowly Changing Dimensions: Type 6**- Combine all types

Customer_ID	Customer_Name	Current_Type	Historical_Type	Start_Date	End_Date	Current_Flag
1	Cust_1	Corporate	Retail	01-01-2010	21-07-2010	N
2	Cust_1	Corporate	Other	22-07-2010	17-05-2012	N
3	Cust_1	Corporate	Corporate	18-05-2012	31-12-9999	Y

1. ~~Slowly Changing Dimensions.~~
2. **CONFORMED DIMENSIONS.**
3. Degenerate Dimensions.
4. Junk Dimensions.
5. Role Playing Dimensions.
6. Static Dimensions.
7. Shrunk Dimensions.
8. Stacked Dimensions.
9. Deferred Dimension.
10. Rapidly Changing Dimensions.

CONFORMED DIMENSIONS (DISTORTED DIMENSIONS)

- In the following example; Geography and Customer Dimensions are being shared by 2 Fact Tables.
 - ✓ Revenue Fact Table stores the revenue generated by both Geography and the Customer.
 - ✓ Product Units Fact Table stores number of units sold in Geography to a Customer.
- Geography and Customer Dimensions are called **CONFORMED DIMENSIONS.**



1. Slowly Changing Dimensions.
2. Conformed Dimensions.
3. **DEGENERATE DIMENSIONS.**
4. Junk Dimensions.
5. Role Playing Dimensions.
6. Static Dimensions.
7. Shrunk Dimensions.
8. Stacked Dimensions.
9. Deferred Dimension.
10. Rapidly Changing Dimensions.

DEGENERATE DIMENSIONS

- Sometimes a dimension is defined that has no content except for its **PRIMARY KEY**.
- A dimension which has ONLY primary key attribute and this attribute can be stored in a fact table and has no associated dimension table is called **DEGENERATE DIMENSION**.
- This **DEGENERATE DIMENSION** is placed in the fact table with the explicit acknowledgment that there is no associated dimension table.
- Degenerate dimensions are most common with **Transaction and Accumulating Snapshot Fact Tables**.

1. Slowly Changing Dimensions.
2. Conformed Dimensions.
3. **DEGENERATE DIMENSIONS.**
4. Junk Dimensions.
5. Role Playing Dimensions.
6. Static Dimensions.
7. Shrunk Dimensions.
8. Stacked Dimensions.
9. Deferred Dimension.
10. Rapidly Changing Dimensions.

- As an example:

- ✓ The table below is fact table with Quantity and Amount as ADDITIVE measures.
- ✓ ProductId comes from Product Dimension.
- ✓ Sometimes, business would want to know the Quantity of products and Amount per **INVOICE**. In this case **InvoiceNumber** is a mandatory attribute in the Fact Table.
- ✓ Invoice Dimension is a dimension with ONLY one attribute InvoiceNumber.
- ✓ In this case; Invoice dimension is DEGENERATED (Ignored) and add the individual attribute directly to the Fact Table.

Invoice Number	ProductId	Quantity	Amount
04182022-1	123	4	200
04182022-1	341	3	100
04192022	124	20	400
04202022	178	1	300

1. ~~Slowly Changing Dimensions.~~

2. ~~Conformed Dimensions.~~

3. ~~Degenerate Dimensions.~~

4. **JUNK DIMENSIONS.**

5. Role Playing Dimensions.

6. Static Dimensions.

7. Shrunk Dimensions.

8. Stacked Dimensions.

9. Deferred Dimension.

10. Rapidly Changing Dimensions.

JUNK DIMENSIONS (FOREIGN DIMENSIONS MANAGER)

- Assume the ATM log is stored in the following form, where there are multiple flags which are set when a transaction T takes place.
 - ✓ If the report generated from the DW shows that most of the transactions failed because there was no cash within the ATM. This could suggest that the cash delivery system is not efficient and corrective steps are needed.
 - ✓ These flags **DO NOT** integrate with the common dimensions like Customer, Product, ... ETC. As these flags are related to the system itself and not to a customer or product.

Successful Tran	Failed	No Cash	Invalid Pin	No Balance
1	0	0	0	0
0	1	1	0	0
0	1	0	1	0
0	1	0	0	1

JUNK DIMENSIONS

(FOREIGN DIMENSIONS MANAGER)

1. ~~Slowly Changing Dimensions.~~
2. ~~Conformed Dimensions.~~
3. ~~Degenerate Dimensions.~~
4. **JUNK DIMENSIONS.**
5. Role Playing Dimensions.
6. Static Dimensions.
7. Shrunk Dimensions.
8. Stacked Dimensions.
9. Deferred Dimension.
10. Rapidly Changing Dimensions.

- ✓ Accommodating all these flags into a fact table will make the number of fields in the fact table very huge and result in performance degradation.
- ✓ Also, every field in the fact table will have a corresponding dimension table (even if it was a Degenerate Dimension). Hence if we have 20 flags, then we need to have 20 different dimensions.
- ✓ Instead of building one dimension for each of these flags, we can combine all the flags and form one single dimension called **JUNK DIMENSION** as below:

JunkDimId	Successful Tran	Failed	No Cash	Invalid Pin	No Balance
1	1	0	0	0	0
2	0	1	1	0	0
3	0	1	0	1	0
4	0	1	0	0	1

1. ~~Slowly Changing Dimensions.~~

2. ~~Conformed Dimensions.~~

3. ~~Degenerate Dimensions.~~

4. **JUNK DIMENSIONS.**

5. Role Playing Dimensions.

6. Static Dimensions.

7. Shrunk Dimensions.

8. Stacked Dimensions.

9. Deferred Dimension.

10. Rapidly Changing Dimensions.

JUNK DIMENSIONS **(FOREIGN DIMENSIONS MANAGER)**

✓ For every combination of these flags, there is a JunkDimId which will be used inside the Fact Table.

✓ This will achieve **THREE** objectives:

1. Reduces the number of fields within the fact table.

2. Saves from creating one dimension for each of the flags.

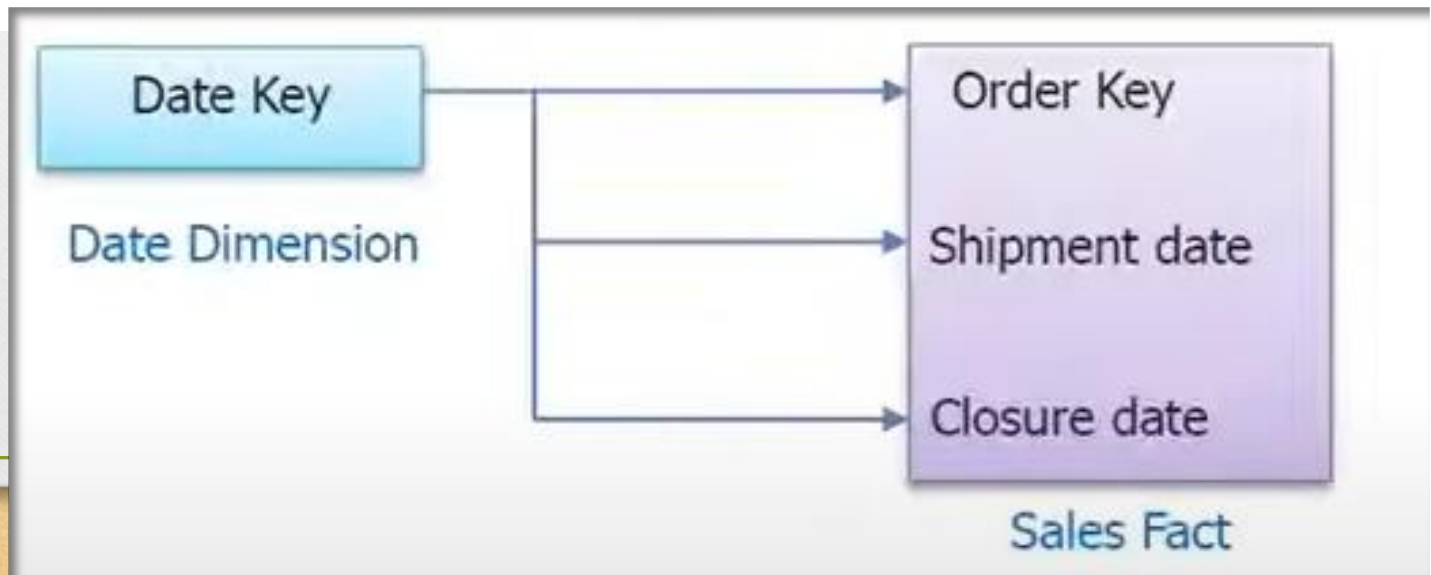
3. Increases the performance.

JunkDimId	Successful Tran	Failed	No Cash	Invalid Pin	No Balance
1	1	0	0	0	0
2	0	1	1	0	0
3	0	1	0	1	0
4	0	1	0	0	1

1. ~~Slowly Changing Dimensions.~~
2. ~~Conformed Dimensions.~~
3. ~~Degenerate Dimensions.~~
4. ~~Junk Dimensions.~~
5. **ROLE PLAYING DIMENSIONS.**
6. Static Dimensions.
7. Shrunk Dimensions.
8. Stacked Dimensions.
9. Deferred Dimension.
10. Rapidly Changing Dimensions.

ROLE PLAYING DIMENSIONS.

- **Role Playing Dimension** is the dimension which may has **MULTIPLE VALID** relationships with the Fact Table.
- As an example, the date dimension can be used to fetch **Order Placed Date**, **Order Shipment Date** or **Order Closure Date**.
- As can be seen below, there are multiple keys in the fact table pointing to the same dimension.



1. ~~Slowly Changing Dimensions.~~

2. ~~Conformed Dimensions.~~

3. ~~Degenerate Dimensions.~~

4. ~~Junk Dimensions.~~

5. **ROLE PLAYING DIMENSIONS.**

6. Static Dimensions.

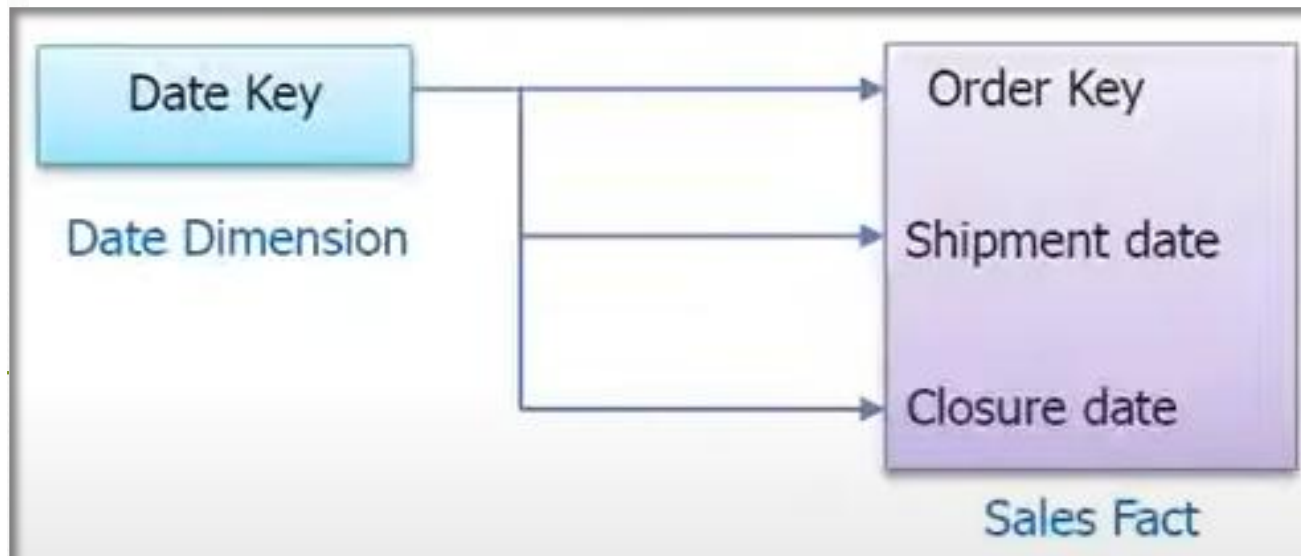
7. Shrunk Dimensions.

8. Stacked Dimensions.

9. Deferred Dimension.

10. Rapidly Changing Dimensions.

ROLE PLAYING DIMENSIONS.



- ✓ In this case we do not have to create 3 Date Dimensions.
- ✓ All the THREE Date References refer to the same Date Dimension.
- ✓ The Date Dimension is stored only once, processed once and accessed THREE times. This saves time and memory space.

1. ~~Slowly Changing Dimensions.~~
2. ~~Conformed Dimensions.~~
3. ~~Degenerate Dimensions.~~
4. ~~Junk Dimensions.~~
5. ~~Role Playing Dimensions.~~
6. **STATIC DIMENSIONS.**
7. ~~Shrunk Dimensions.~~
8. ~~Stacked Dimensions.~~
9. ~~Deferred Dimension.~~
10. ~~Rapidly Changing Dimensions.~~

STATIC DIMENSIONS. (FIXED DIMENSION)

- **Static Dimensions** are those dimensions which are not extracted from the original data source but are created within the context of the data warehouse.
- **Static Dimensions** can be:
 - ✓ **Loaded Manually** - for example with status codes.
 - ✓ **Generated By a Procedure**, such as a date or time dimension
- **As an example, the Date, Geography or Location Dimensions.**

1. ~~Slowly Changing Dimensions.~~
2. ~~Conformed Dimensions.~~
3. ~~Degenerate Dimensions.~~
4. ~~Junk Dimensions.~~
5. ~~Role Playing Dimensions.~~
6. ~~Static Dimensions.~~
7. **SHRUNKEN DIMENSIONS.**
8. Stacked Dimensions.
9. Deferred Dimension.
10. Rapidly Changing Dimensions.

SHRUNKEN DIMENSIONS.

- A **Shrunken Dimension** is a subset of another dimension.
- For example, the orders fact table may include a foreign key for product, but the target fact table may include a foreign key only for Product Category, which is in the product table, but much less granular.
- Creating a smaller dimension table, with Product Category as its primary key, is one way of dealing with this situation of heterogeneous grain.
- If the product dimension is snowflaked, there is probably already a separate table for Product Category, which can serve as the **shrunk dimension**

1. ~~Slowly Changing Dimensions.~~
2. ~~Conformed Dimensions.~~
3. ~~Degenerate Dimensions.~~
4. ~~Junk Dimensions.~~
5. ~~Role Playing Dimensions.~~
6. ~~Static Dimensions.~~
7. ~~Shrunk Dimensions.~~
8. **STACKED DIMENSIONS.**
9. Deferred Dimension.
10. Rapidly Changing Dimensions.

STACKED DIMENSIONS

- A **Stacked Dimension** is used where two or more dimensions are combined, and their attributes are Fact Table Properties and not dimensions properties.
- We see many types and status columns: Product Type, Customer Status, Store Type, Security Type, Security Class, Broker Type, etc. All the columns store to their respective dimensions because they are the properties of the dimension.

TransactionTypeKey	TransactionType	Category
0	Unknown	Unknown
1	Direct sale	Transaction type
2	Refund	Transaction type
3	Purchase	Transaction type
4	eCommerce	Transaction type
5	Down payment	Payment type
6	Full delivery	Payment type
7	On delivery	Payment type

- ~~1. Slowly Changing Dimensions.~~
- ~~2. Conformed Dimensions.~~
- ~~3. Degenerate Dimensions.~~
- ~~4. Junk Dimensions.~~
- ~~5. Role Playing Dimensions.~~
- ~~6. Static Dimensions.~~
- ~~7. Shrunk Dimensions.~~
- ~~8. Stacked Dimensions.~~
9. **DEFERRED DIMENSION.**
10. Rapidly Changing Dimensions.

DEFERRED DIMENSION

INFERRED DIMENSIONS

- When loading a fact record, a dimension record may not be ready yet.
- One solution is to generate a surrogate key with null for all the other attributes. This should technically be called an inferred member but is often called an inferred dimension.
- It is technically called an inferior member but is often called as sensible dimension.
- A fact table that references dimension members that are not yet loaded contains inferred members.
- Data for inferred members can be updated rather than created when it is loaded, so you do not need to create a new record.
- **A late-arriving dimension** (sometimes called **early-arriving facts**) occurs when the dimension data arrives in the data warehouse later than the fact data that references it.

1. ~~Slowly Changing Dimensions.~~
2. ~~Conformed Dimensions.~~
3. ~~Degenerate Dimensions.~~
4. ~~Junk Dimensions.~~
5. ~~Role Playing Dimensions.~~
6. ~~Static Dimensions.~~
7. ~~Shrunk Dimensions.~~
8. ~~Stacked Dimensions.~~
9. **DEFERRED DIMENSION.**
10. Rapidly Changing Dimensions.

DEFERRED DIMENSION

INFERRED DIMENSIONS

Promo Code PK	Promo Code	Promotion Type	Discount Type	Inferred
1	P1	Marketing	Cashback	0
2	P2	Employee	Instant	0
3	Pspecial	Unknown	Unknown	1

TicketId	Transaction Date	Promo Code PK	Quantity
T1	04172022	1	100
T2	04182022	2	200
T3	04202022	3	700

1. Slowly Changing Dimensions.

2. Conformed Dimensions.

3. Degenerate Dimensions.

4. Junk Dimensions.

5. Role Playing Dimensions.

6. Static Dimensions.

7. Shrunk Dimensions.

8. Stacked Dimensions.

9. **DEFERRED DIMENSION.**

10. Rapidly Changing Dimensions.

Employee

Team

Transaction System

OLAP System

DEFERRED DIMENSION

INFERRED DIMENSIONS

EmpId	Name	Gender	Salary
1	Ahmed	Male	10000
22	Maye	Female	12000

TeamId	Name	EmpId
1	Sales	1
3	Manufacturing	12

Id	ProductId	CustId	EmpId	Amount
1	P1	1	1	1000
2	P2	2	22	100

Id	ProductId	CustId	EmpId	TeamID	Amount
1	P1	1	1	1	1000
2	P2	2	22	Null	100

Referential Integrity Constraints Violations

~~1. Slowly Changing Dimensions.~~

~~2. Conformed Dimensions.~~

~~3. Degenerate Dimensions.~~

~~4. Junk Dimensions.~~

~~5. Role Playing Dimensions.~~

~~6. Static Dimensions.~~

~~7. Shrunk Dimensions.~~

~~8. Stacked Dimensions.~~

9. **DEFERRED DIMENSION.**

10. Rapidly Changing Dimensions.

DEFERRED DIMENSION

INFERRED DIMENSIONS

FOUR solutions:

1. **Never Process the Fact (Ignore records NULL value for the FKs) .**
2. Park the record and retry.
3. Insert a dummy value for the missing assignment. (90% of the DW folks do this choice) .
4. Insert a record in the dimension and update (fill in the missing values) later.

Id	ProductId	CustId	EmpId	TeamID	Amount
1	P1	1	1	1	1000
2	P2	2	22	Null	100

1. Slowly Changing Dimensions.
2. Conformed Dimensions.
3. Degenerate Dimensions.
4. Junk Dimensions.
5. Role Playing Dimensions.
6. Static Dimensions.
7. Shrunk Dimensions.
8. Stacked Dimensions.
9. **DEFERRED DIMENSION.**
10. Rapidly Changing Dimensions.

DEFERRED DIMENSION

INFERRED DIMENSIONS

FOUR solutions:

1. ~~Never Process the Fact (Ignore records NULL value for the FKs).~~
2. **Park the record and retry.**
3. Insert a dummy value for the missing assignment. (90% of the DW folks do this choice).
4. Insert a record in the dimension and update (fill in the missing values) later.

Id	ProductId	CustId	EmpId	TeamID	Amount
1	P1	1	1	1	1000

Team

TeamId	Name	EmpId
1	Sales	1
3	Manufacturing	12
4	Sales-2	22

Id	ProductId	CustId	EmpId	TeamID	Amount
1	P2	2	22	4	100

LANDING AREA

1. ~~Slowly Changing Dimensions.~~

2. ~~Conformed Dimensions.~~

3. ~~Degenerate Dimensions.~~

4. ~~Junk Dimensions.~~

5. ~~Role Playing Dimensions.~~

6. ~~Static Dimensions.~~

7. ~~Shrunk Dimensions.~~

8. ~~Stacked Dimensions.~~

9. **DEFERRED DIMENSION.**

10. ~~Rapidly Changing Dimensions.~~

DEFERRED DIMENSION

INFERRED DIMENSIONS

FOUR solutions:

1. ~~Never Process the Fact (Ignore records NULL value for the FKs).~~

2. ~~Park the record and retry.~~

3. **Insert a dummy value for the missing assignment. (90% of the DW folks do this choice).**

4. Insert a record in the dimension and update (fill in the missing values) later.

Team

TeamId	Name	EmpId
1	Sales	1
3	Manufacturing	12
-1	Dummy	22

Id	ProductId	CustId	EmpId	TeamID	Amount
1	P1	1	1	1	1000
2	P2	2	22	-1	100

1. Slowly Changing Dimensions.

2. Conformed Dimensions.

3. Degenerate Dimensions.

4. Junk Dimensions.

5. Role Playing Dimensions.

6. Static Dimensions.

7. Shrunk Dimensions.

8. Stacked Dimensions.

9. **DEFERRED DIMENSION.**

10. Rapidly Changing Dimensions.

DEFERRED DIMENSION

INFERRED DIMENSIONS

FOUR solutions:

1. ~~Never Process the Fact (Ignore records NULL value for the FKs).~~

2. ~~Park the record and retry.~~

3. ~~Insert a dummy value for the missing assignment. (90% of the DW folks do this choice).~~

4. Insert a record in the dimension and update (fill in the missing values) later.

Team

TeamId	Name	EmpId
1	Sales	1
3	Manufacturing	12
4	Unknown	22

Id	ProductId	CustId	EmpId	TeamID	Amount
1	P1	1	1	1	1000
2	P2	2	22	4	100

1. ~~Slowly Changing Dimensions.~~

2. ~~Conformed Dimensions.~~

3. ~~Degenerate Dimensions.~~

4. ~~Junk Dimensions.~~

5. ~~Role Playing Dimensions.~~

6. ~~Static Dimensions.~~

7. ~~Shrunk Dimensions.~~

8. ~~Stacked Dimensions.~~

9. ~~Deferred Dimension.~~

10. **RAPIDLY CHANGING DIMENSIONS.**

RAPIDLY CHANGING DIMENSIONS.

- A dimension is a fast changing or rapidly changing dimension if one or more of its attributes in the table changes very fast and in many rows. Handling rapidly changing dimension in data warehouse is very difficult because of many performance implications.
- As you know slowly changing dimension type 2 is used to preserve the history for the changes. But the problem with type 2 is, with each change in the dimension attribute, it adds a new row to the dimension.
- If in case there are dimensions that are changing a lot, table becomes larger and may cause serious performance issues. Hence, use of the type 2 may not be the wise decision to implement the rapidly changing dimensions.

1. ~~Slowly Changing Dimensions.~~

2. ~~Conformed Dimensions.~~

3. ~~Degenerate Dimensions.~~

4. ~~Junk Dimensions.~~

5. ~~Role Playing Dimensions.~~

6. ~~Static Dimensions.~~

7. ~~Shrunk Dimensions.~~

8. ~~Stacked Dimensions.~~

9. ~~Deferred Dimension.~~

10. **RAPIDLY CHANGING DIMENSIONS.**

RAPIDLY CHANGING DIMENSIONS.

- For example: Consider patient dimension where there are 1000 rows in it. On average basis, each patient changes the 10 of attributes in a year. If you use the type 2 to manage this scenario, there will be $1000 \times 10 = 10000$ rows. Imagine if the table has 1 million rows, it'll become very hard to handle the situation with type 2. Hence, we use rapidly changing dimension approach.
- **Separate Rapidly Changing Attribute by Implementing Junk Dimension**
- Consider the fact table, in which not all the attributes of the table changes rapidly. There may be some attribute that may be changing rapidly and other not. The idea here is to separate the rapidly changing attribute from the slowly changing ones and move those attribute to another table called junk dimension and maintain the slowly changing attribute in same table. In this way, we can handle situation of increasing table size.

1. ~~Slowly Changing Dimensions.~~

2. ~~Conformed Dimensions.~~

3. ~~Degenerate Dimensions.~~

4. ~~Junk Dimensions.~~

5. ~~Role Playing Dimensions.~~

6. ~~Static Dimensions.~~

7. ~~Shrunk Dimensions.~~

8. ~~Stacked Dimensions.~~

9. ~~Deferred Dimension.~~

10. **RAPIDLY CHANGING DIMENSIONS.**

RAPIDLY CHANGING DIMENSIONS.

Consider the **PATIENT** table with following attributes:

- PatientId
- Name
- Gender
- MaritalStatus
- Weight
- BMI

PATIENT
Patient_id
Name
Gender
Marital_status

PATIENT_JNK_DIM
Pat_SK
Weight
BMI

1. ~~Slowly Changing Dimensions.~~
2. ~~Conformed Dimensions.~~
3. ~~Degenerate Dimensions.~~
4. ~~Junk Dimensions.~~
5. ~~Role Playing Dimensions.~~
6. ~~Static Dimensions.~~
7. ~~Shrunk Dimensions.~~
8. ~~Stacked Dimensions.~~
9. ~~Deferred Dimension.~~
10. **RAPIDLY CHANGING DIMENSIONS.**

RAPIDLY CHANGING DIMENSIONS.

- **Link Junk Dimension and PATIENT Table:** In this step, we must link the junk dimension and patient table.
- Keep in mind; we cannot simply refer the junk dimension table by adding its primary key to patient table as foreign key.
- Because any changes made to junk dimension will have to reflect in the patient table, this obviously increases the data in patient dimension.
- Instead, we create one more table called mini dimension that acts as a bridge between Patient and Junk dimension.
- We can also add the columns such as start and end date to track the change history. Below is the structure of the mini dimension:

PATIENT
PatientId
Name
Gender
MaritalStatus



PATIENT_MINI_DIM
PatientId
PatSK
START_DATE
END_DATE



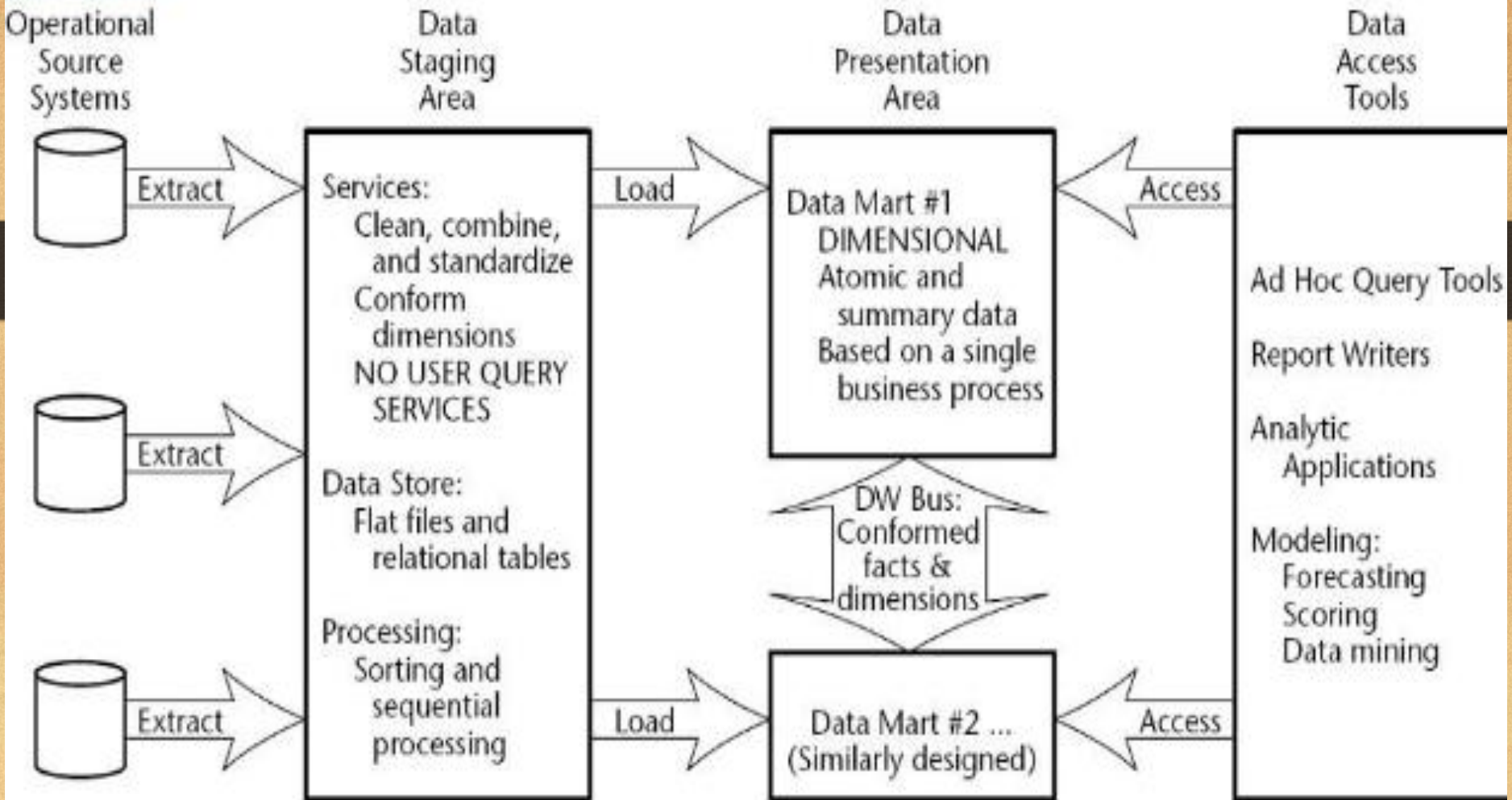
PATIENT_JN K_DIM
PatSK
Weight
BMI



PATIENT_JNK_D IM
<u>PatSK</u>
Weight
BMI
<u>StartDate</u>
<u>EndDate</u>

Components of a DW

DW: **Data Warehousing**



Components of a DW

- **Data sources.**
- **Data staging Area (DSA) / Operational Data Store (ODS)**
- **Data Marts/Data Warehouse.**
- **Presentation Area.**
- **Data Access Tool.**

DW Components

Operational Source Systems (Data Sources)

- These are the operational systems of record that capture the transactions of the business.
- Should be thought of as outside the DW.
- The main **priorities** of the sources systems are performance and availability.
- The sources maintain little historical data. But IF you have a good data warehouse; **then no need for the data sources to hold history or at least to hold long time interval of historical data.**

Components of a DW

Data staging Area (DSA) / Operational Data Store (ODS)

- Is both a storage area and an operational area for a set of processes commonly referred as **Extract** → **Transform** → **Cleanse** → **Load** (**ETL/ ETCL**)
- The DSA/ODS is everything between the Data Marts/DW and the data presentation area / layer.

DW Components

Extraction (E)

-
- After setting up the business objectives of the DW or the Subject Statement.
 - Extraction is the first step in the process of getting data into the data warehouse environment.
 - Extraction means reading and understanding the source data and copying the data needed for the data warehouse into the staging area for further data manipulation.
 - Used methods for extraction:
 - Get the whole table every time.
 - Get the table incrementally (ONLY the new additions / Changes).
 - Using a fixed time periods.

DW Components

Extraction (E)

- Which method to use? **Different considerations**
 - **The Time Window taken to extract the data from the operational data sources:** the data extraction process should be completed in a certain amount of time. The less time it requires to do the data extraction, the better it is for a DW system. Because this means that DW will be updated more frequently.
 - **The second consideration is practically.** I.e., What are possible and what are not.
 - We need to extract data incrementally, but this may not be possible then we should search for and use another method.
 - We need to extract the whole table but if the table has 100 millions records, then we may have to do it incrementally (if possible)

DW Components

Extraction (E)

- **Whole Table Every Time?**

- The simplest way to download a table from an operational data source / source system into a data warehouse is to extract all records every time using **SELECT * FROM Datasource1.Table**
 - If a company has 500 **branch**
 - Each **branch** has a record and each record stores:
 - Name, Address and other attributes.
 - For 500 records, even if the **branch** table has **timestamp for both create and update**, it is probably quicker to get the entire table every time.

DW Components

Extraction (E)

- **Whole Table Every Time?**

- If we do a **SELECT** without a **WHERE** clause, the extraction process starts immediately/instantly. If the **WHERE** clause is filled by at least one condition e.g. Timestamp, the extraction process begins a few moments later based on:
 - How many records on the base table.
 - Whether the columns used for the:
 - ✓ **WHERE**
 - ✓ **SELECT** are indexed or not.
 - The type of index used (in case of indexing) → Primary index, Secondary/non-clustered index and/or Coverage Index
 - The capabilities of the used machine for the extraction process.

- **ONE** way to determine if the whole table download every time is a good choice or not is to roughly estimate the time needed for the extraction process.

- If the **WHOLE TABLE EVERY TIME EXTRACTION** is not possible and does not cope with the time associated to the process (time constraints/standards) and/or business requirements, then we extract the data **INCREMENTALLY (IF POSSIBLE, ALSO)**.
- Otherwise, the **WHOLE TABLE EVERY TIME** is used.
- Here the slowly changing dimensions and rapidly changing dimensions are determined and the way is determined.

DW Components

Extraction (E)

- **Incremental EXTRACT**

- If the table contains 2 million records, say it is the customer table, it probably takes 15- 20 Minutes (or more based on the answers to the above questions and the capability of the machine used) to download the whole table, if there are timestamp columns for creation and updates, then it is sensible data to try to download this table incrementally which reduces the download time to 3-5 minutes. Roughly the number of records decreases from 2 million records to 2000 (specific period changes ONLY E.g., Daily Changes ONLY).
- 5 Well known options to perform the incremental extract:
 - Using the Timestamp Columns.
 - Using Triggers.
 - Using the Transaction Date.
 - Using the Data Partitioning.
 - Mixed Approach (Combination of the above 4 ways).

DW Components

Extraction (E)

- **Incremental Extract Using Timestamp columns:**
 - **SELECT * FROM** DataSource1.Table **WHERE** DateColumn > **LSET**
 - **LSET** → Last Successful Extract Time.
 - Before the extract begins, the data warehouse system time is stored in a variable:
 - If the extract is **successful**, we store this variable into the DW control database (MetaData) as **LSET**, so it will be used to conduct next EXTRACT runs.
 - Else if the extract process **FAILs**, it is important to **NOT** store this variable into the DW control database (MetaData). So that **NEXT TIME WHEN THE** extract runs, it uses the **LSET** stored, and gets the records since the last successful extract.

DW Components

Extraction (E)

- **Incremental Extract Using Triggers:**
 - Triggers are defined ON the source system tables to keep the PKs (**Identifiers**) of the changed records (added and/or updated) in a dedicated different table **NAMED DELTA TABLE**.
 - The **DELTA TABLE** needs to be cleared out when it **SUCCESSFULLY** processed.

DW Components

Extraction (E)

- **Incremental Extract Using Transaction Date:**

- If the table processed and extracted is transactional table (contains dated transactions), it may contain more than one column to represent the date (this incremental method is valid to be applied here).
- Else if the table does not have a date BUSINESS columns, then the METADATA date columns may be used to conduct this purpose. E.g. Last Updated Date, Last Insertion Date.

DW Components

Extraction (E)

- **Incremental Extract Using Data Partitioning:**

- If a partitioning strategy is applied over the table downloaded or extracted, then this method is applicable, otherwise DW folks may recommend this to the operational systems folks.
- Using partitioning by range over time.
 - Extract records that fall within a specific time range (partition).
 - Partitioning data by range over time helps identifying the data for the current week.

DW Components

Transformation (T)

- **Once the data is extracted to the ODS/DSA, there are a set of protentional transformations, such as:**
 - Convert between units to match the target schema (DW Schema) used units → This serves the process of combining data from multiple sources.
 - Solve the duplicates (very complex process) especially for the **NON-Federated** data sources.
 - Standardize the formats (e.g. DateTime Formats).
 - Assign keys when integrating data from sources where **SOME OR ALL** of them are not federated.
 - Cleanse the data (Correct misspellings, resolving domain conflicts, **dealing with missing elements, ...**) [Fill in missing values, resolve noise data, handle the outliers especially if they represent noise data]
 - **Detect the duplicates/ conflicts.**
 - Apply data fusion.

DW Components

Transformation (T) → Cleanse (C)

■ Data quality problems appear over:

- **Single data source problems:**
 - ✓ Schema Problems (E.g. lack of integrity constraints, poor schema design,...)
 - ✓ Instance Problems (E.g. data entry errors)
- **Multiple data source problems (problems on integration process):**
 - ✓ Schema Problems (E.g. heterogeneous data models and schema design,...)
 - ✓ Instance Problems (E.g. different representations, different measurement system ...)

■ So data sources may conflict with each other over three levels:

- Schema Level Inconsistencies,
- Representation Level Inconsistencies, and
- Data Level Inconsistencies

DW Components

Transformation (T)

- ▶ Data quality problems after the integration process
- ▶ If we integrated complete and accurate data sources 2 main problems are appeared after the integration process:
 - **Mutual Inconsistency/Data Conflict:** multiple representations of same real-world object with conflicts between their data values (redundancy with different data values for the same object).
 - **Incompleteness:** Incomplete set as an answer to the user query (Less records than expected and/or existing or even MORE records than expected and/or existing)

DW Components

Transformation (T)

- **Three well defined strategies to handle conflicts:**
 - **Conflict Ignorance (CI) Strategy:** ignore handling with conflicts
 - **Conflict Avoidance (CA) Strategy :** Detect duplicates and avoid dealing with conflicts by taking a pre-decision to handle conflicts, this decision may be
 - Metadata based decision, or
 - Content-based decision.
 - **Conflict Resolution (CR) Strategy :** Detect duplicates and try to resolve them
 - Metadata based decision, or
 - Content-based decision

Conflict Handling Strategy/Step	Conflict Detection	Conflict Resolution
Conflict Ignorance	X	X
Conflict Avoidance	√	X
Conflict Resolution	√	√

DW Components

Transformation (T)

- Employees integrated from different data sources
- Extraction query gets ID, Name, Address, Zip_Code of employees from different data sources and applied the MAPPING required.
- And the data retrieved during the user query is:

ID	Name	Address	Zip_Code
1234567	Dandarawy	Xyz	1234
1234567	Dandarawy	Yxz	2341
112	Dandarawy	Xyz	1234

DW Components

Transformation (T)

ID	Name	Address	Zip_Code			
1234567	Dandarawy	Xyz	1234			
1234567	Dandarawy	Yxz	2341			
112	Dandarawy	Xyz	1234			
ID	Name	Address	Zip_Code	Source	Confidence	Timestamp
1234567	Dandarawy	Xyz	1234	S1	0.9	0.8
1234567	Dandarawy	Yxz	2341	S2	0.6	0.9

Data fusion is the process of combining these conflicting records into **COMPLETE & ACCURATE** Record. Instance based and MetaData Based Resolution/Fusion.

Extraction Query Q(Name, Address, Postal_Code, DoB)

**What is the data to be used when load the data to DW ???
T1, T2, or Both**

Islamic_Customers

ID	Name	Gender	Address	Postal_Code	DoB
1	Ali Zidane	1	XYFayoum	123456	11/8/1985
2	Aly Zedan	1	MNCairo	123458	NULL

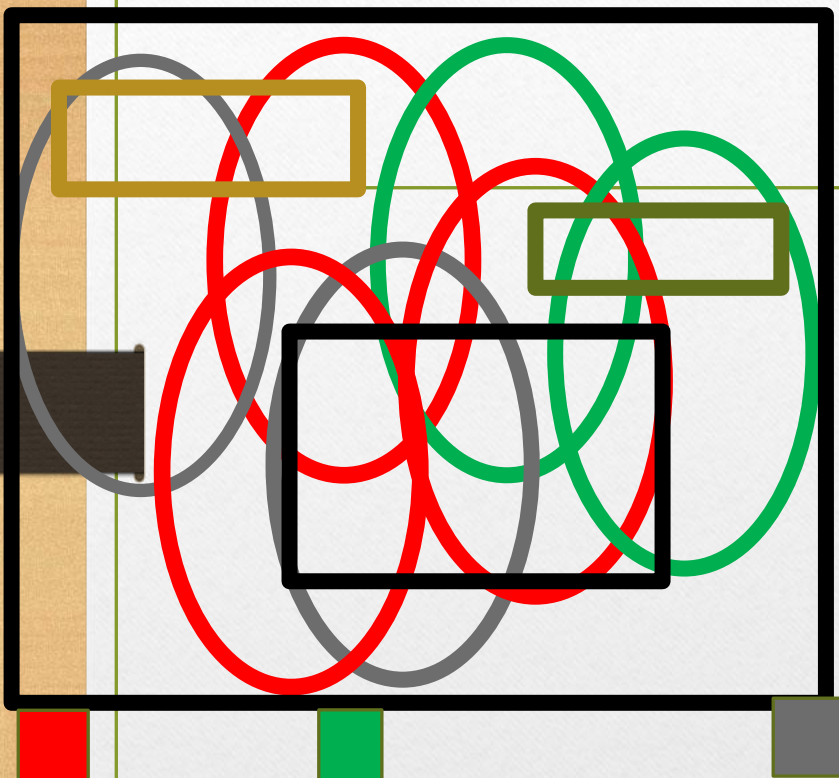
Customer

Islamic Customers

ID	Name	Gender	Address	Postal_Code	DoB	Type
1	Ali Zidane	1	XYFayoum	123456	11/8/1985	1
2	Ali Zidane	1	MNCairo	123458	11/8/19985	0

Customer

CID	Name	Gender	Address	Postal_Code	Type
2	Aly Zedan	1	MNCairo	123458	1
34	Noha Zidane	0	MMFayoum	345789	0



- **Different Representations.**
- **Different Identifiers/MatcherSets.**
- **Different Mappings.**
- **Duplicated entities, and due to duplicates inconsistencies produced.**
- **Different lifetime states for the same duplicated real-world entity with inconsistent timestamps or even absent.**

$O(X, Y, Z)$

$O_o(X, M, N, T)$

$Obj(Y, Z, N, T, F)$

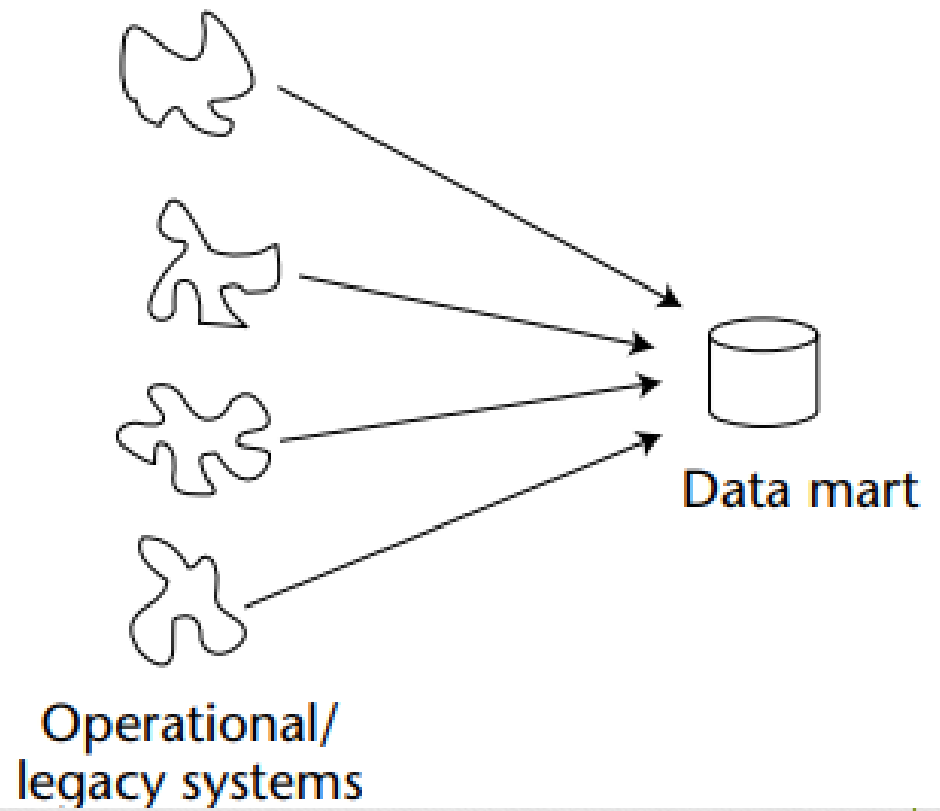
DW Components

Loading (L)

-
- ▶ Loading is the final step of the ETCL processes.
 - ▶ Loading in the DW environment usually takes the form of presenting the quality assured data to the bulk/batch loading facilities of each data mart.
 - ▶ The target data mart must then index the newly arrived data for query performance.

Independent Data Marts

The **Independent Data Mart** is a data mart that is built directly from the legacy applications.



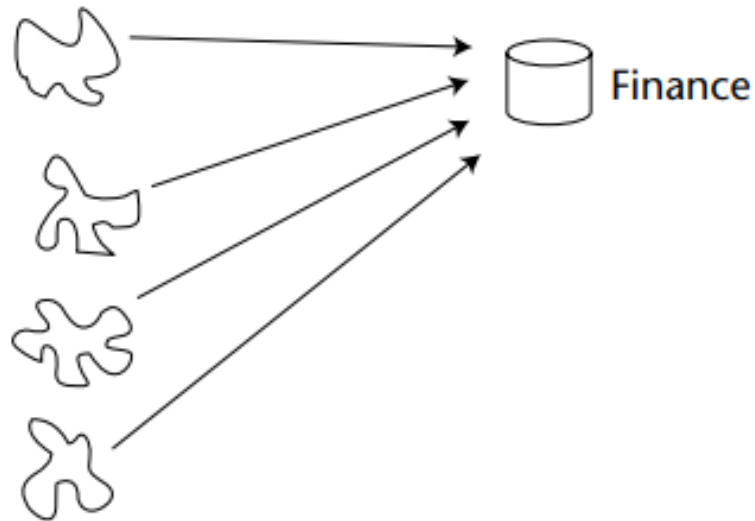
Independent Data Marts

Independent data marts are good for short, fast solutions, but when the longer-term perspective is taken, Independent Data Marts are simply inadequate to solve the problems of information across the organization

Building Independent Data Marts: Pros. & Cons.

- Pros already detailed.
- Cons are shown below:

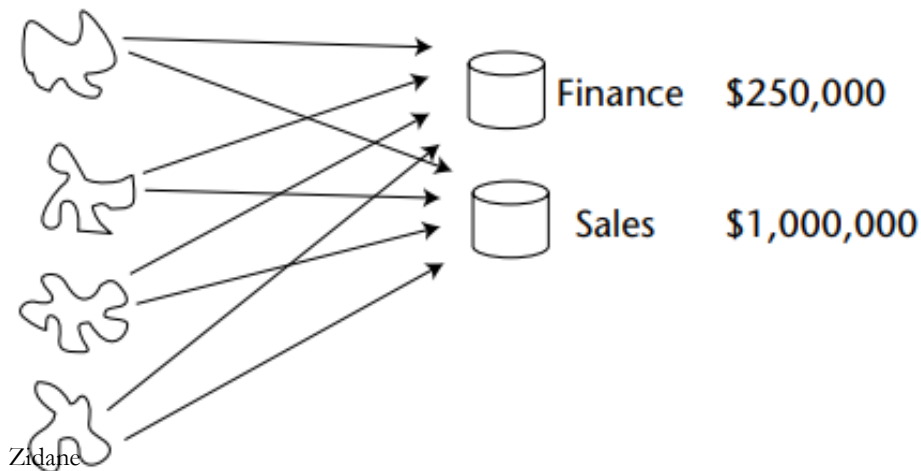
1. If there is only one data mart
→ There is no problem, but this is not always the case.



The first independent data mart is successful.

Building Independent Data Marts: Pros. & Cons.

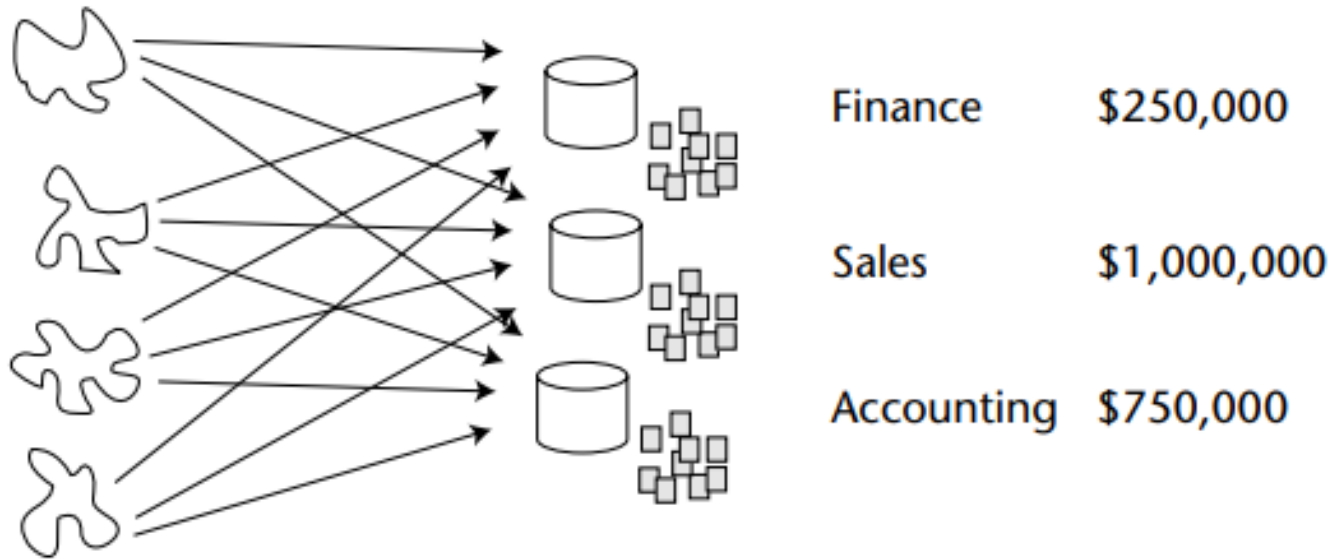
1. The Information between the two **Data Marts** is not Integrated and is not Synchronized.
2. **Same query to the two data marts give different answers.**



The second independent data mart is also successful. But someone notices that the answer to similar questions is not coordinated.

Building Independent Data Marts: Pros. & Cons.

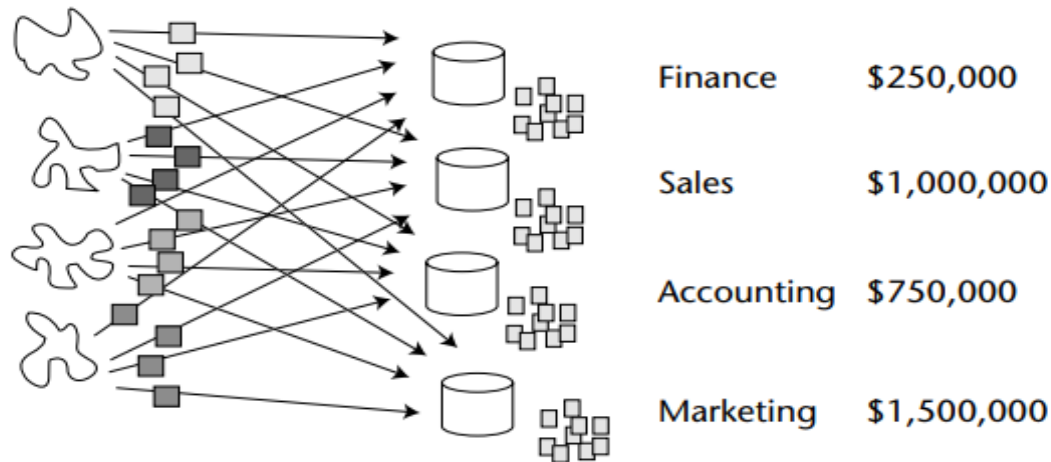
1. The Information between the two Data Marts is not Integrated and is not Synchronized.
2. Same query to the two data marts give different answers.
3. **With each of the Independent Data Marts, there is an increasing redundancy of detailed data. ONE COPY of some or most of the data with each new data mart added (new business comes in).**



 Zidane The third data mart is added. Now there are three opinions on everything and the same detailed data appears every time a new data mart is added.

Building Independent Data Marts: Pros. & Cons.

1. The Information between the two Data Marts is not Integrated and is not Synchronized.
2. Same query to the two data marts give different answers.
3. With each of the Independent Data Marts, there is an increasing redundancy of detailed data
4. **Too many interfaces and UI acting with data marts need to be built and maintained.**

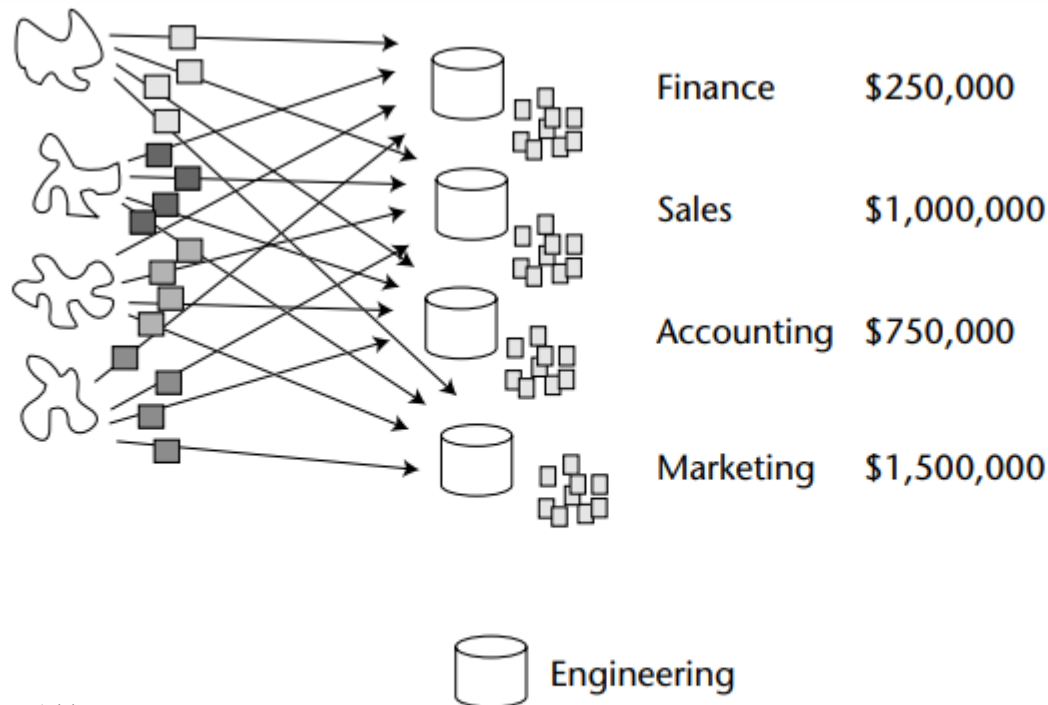


Zidane

The fourth data mart is added. There is yet one more option that cannot be correlated with any other opinion. There is more detailed data that is gathered and is redundant. There are a whole host of interface programs that have to be built and maintained.

Building Independent Data Marts: Pros. & Cons.

1. The Information between the two Data Marts is not Integrated and is not Synchronized.
2. Same query to the two data marts give different answers.
3. With each of the Independent Data Marts, there is an increasing redundancy of detailed data.
4. Too many interfaces and UI acting with data marts need to be built and maintained.
5. **New data mart(s) redoes the same work or very similar one more time which increases the other cons. Especially if none of the ready components are reusable**

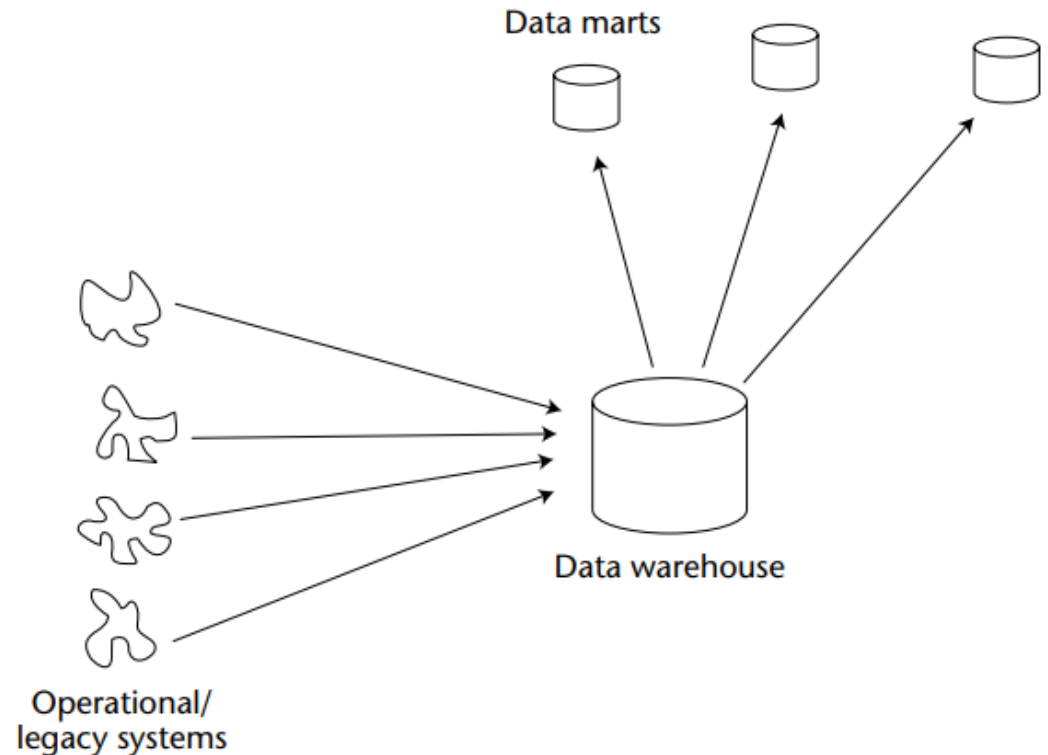


Zidane

Engineering wishes to build a data mart. There is no foundation on which to build and there is no historical data to access. All integration must be redone (one more time), and there is no coordination with any previous efforts at integration.

Dependent Data Marts

The opposite of an Independent Data Mart is a Dependent Data Mart. A Dependent Data Mart is one that is built from data coming from the data warehouse.



Dependent Data Marts are simply adequate to solve the problems of information across the organization

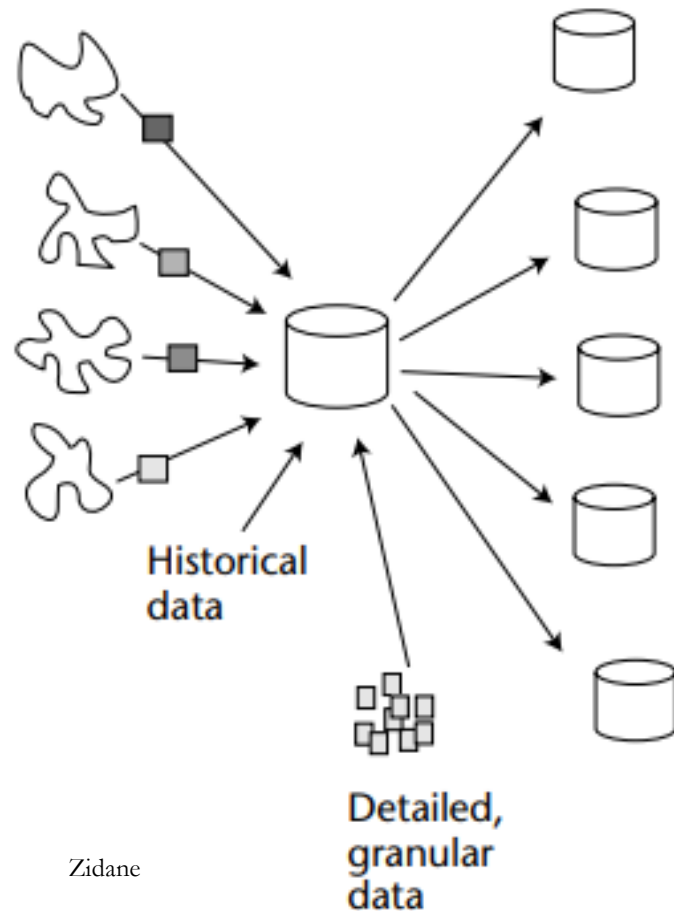
Zidane

- The **Dependent Data Mart** does not depend on legacy or operational data for its source.
- **Dependent Data Mart** depends on only the data warehouse for its source data.
- The **Dependent Data Mart** requires forethought and investment. It requires that someone “think globally.”
- The **Dependent Data Mart** requires multiple users to pool their information needs for the creation of the data warehouse.
- In other words, the **Dependent Data Mart** requires advance planning, a long-term perspective, global analysis, and cooperation and coordination of the definition of requirements among different departments of an organization.

What are Dependent Data Marts

Dependent Data Marts

With dependent data marts, all problems of independent data marts are solved.



Zidane

With dependent data marts, all problems of independent data marts are solved.

To Summarize ...

- The main difference between **Dependent & Independent Data Marts** is that the **Dependent Data Marts** get data from an already created data warehouse while the **Independent Data Marts** get data directly from an operational source and/or external source.

To Summarize ...

- In brief, a data warehouse is a system that helps to analyze data, create reports and visualize them to make business decisions
- An organization has multiple data sources
- Data warehouses collect this data and help to analyze them to take business insights.
- A **Data Mart** is a subset of data stored in a data warehouse. These **Data Marts** provide more security and data integrity.
- Furthermore, there are two types of data marts as **Dependent & Independent Data Marts**.

To Summarize ...

The problems with the **Independent Data Marts** are that the independent data marts

1. Do not provide a platform for reusability
2. Do not provide a basis for reconciliation of data
3. Do not provide a basis for a single set of legacy interface programs
4. Do require that every **Independent Data Mart** create its own pool of detailed data, which is, unfortunately, massively redundant with the pools of detailed data created by other **Independent Data Marts**.
5. The good thing is that; **Dependent Data Marts** that take data from a data warehouse do not have to have the same set of architectural problems.

To Summarize ...

- **Dependent VS Independent Data Marts**
 - **Methods of obtaining data:** **Dependent Data Marts** get data from a central data warehouse that has already been created. However, **Independent Data Marts** get data directly from operational sources or external sources of data or both.
 - **Functionality:** **Dependent Data Marts** obtain data by aggregating, restricting and summarizing the data of the data warehouse **Independent Data Marts** get data from multiple transaction systems in one subject area or department to support specific business needs. Hence, this is a major difference between **Dependent & Independent Data Marts**.
 - **Dependability:** **Dependent Data Marts** depend on the central data warehouse while **Independent Data Marts** are standalone data marts and do not depend on the central data warehouse.
 - **Security:** Another difference between **Dependent & Independent Data Marts** is that the **Independent Data Marts** are more secure than **Dependent Data Marts**.

To Summarize ...

- **Dependent VS Independent Data Marts**
 - Control on Data: One major difference between **Dependent & Independent Data Marts** based on control. **Independent Data Marts** have more control over their data than **Dependent Data Marts**.

Conclusion:

- In brief, a data mart is a subset of a data warehouse that focuses on a single functional area or an organization.
- There are two types of data marts as **Dependent & Independent Data Marts**.
- The difference between **Dependent & Independent Data Marts** is that the **Dependent Data Marts** get data from already created data warehouse while the independent data marts get data directly from an operational, external source or both.

Hybrid Data Marts

Hybrid Data Marts

- A **Hybrid Data Mart** combines data from an existing data warehouse and other operational source systems.
- It unites the speed and end-user focus of a top-down approach with the benefits of the enterprise-level integration of the bottom-up method.